

POLER-OS

Технический архитектурный аудит

Файловая система POLER-FS / Сравнение с 12 ФС / Юридический анализ /
Совместимость Windows / Античиты / Roadmap / Критический аудит

Версия 1.0 / Июль 2026

1. POLER-FS: Технический аудит файловой системы

Данный раздел содержит глубокий технический аудит предлагаемой файловой системы POLER-FS. Анализ критически оценивает каждое архитектурное решение и предлагает улучшения там, где текущий дизайн не соответствует современным требованиям.

1.1 Общая архитектура: Block Groups vs Extent-based

Предложенная архитектура block groups заимствована из ext2/3/4. Для ФС, заявленной как современная и NVMe-first, это **фундаментально ошибочный выбор**. Block groups создают фиксированное число inode в группе, приводят к фрагментации, требуют $O(n)$ сканирования bitmap и не масштабируются до 100TB+ томов.

Рекомендуемая альтернатива — **Extent-based архитектура с B+ деревом**. Каждый файл описывается набором extent-ов (логический диапазон файлов, физический диапазон на диске, длина, флаги). Extent-ы хранятся в B+ дереве внутри inode (inline root) и при переполнении — в отдельных блоках. Это даёт: минимума метаданных для больших файлов, естественную поддержку sparse files, совместимость с CoW и NVMe range I/O.

Критерий	Block-based (предложено)	Extent-based (рекомендуется)
Фрагментация метаданных	Высокая	Низкая
Метаданные для 100GB файла	~200KB indirect blocks	~4KB extent tree
Random read	2-3 IOPS на блок	1 IOPS (extent-LBA)
Sparse files	Требует спец. флагов	Естественная поддержка
CoW совместимость	Плохая	Отличная
NVMe оптимизация	Ограниченная	Прямая (extent-range)

POLER-FS ДОЛЖЕН быть extent-based. Block-based подход безнадёжно устарел. Ext4 сама перешла на extents в 2008 году.

1.2 Суперблок

Один 4KB суперблок без резервных копий — **критически недостаточно**. Предложенный дизайн не содержит UUID тома, feature flags, версии on-disk формата, контрольных сумм. Без этих полей ФС нежизнеспособна в продакшене.

Рекомендуемый суперблок должен содержать: 128-bit UUID тома, метку тома (UTF-16LE для NT compat), геометрию (block/sector/page/cluster size), позицию и размер inode таблицы (как extent map), позицию и размер журнала, feature flags (compat/ro_compat/incompat/special), Wrapped KEK для шифрования, алгоритм контрольной суммы, BLAKE3 хеш суперблока.

Резервные копии: минимум 5 копий — блок 0 (primary), блок 1 (backup #1), N/1024 (промежуточные), конец тома (backup #4-5). При загрузке — читаем primary, проверяем checksum, при failed — сканируем все копии.

1.3 Организация Inode

256 байт — **критически мало** для inode, который должен содержать NT ACL, шифрование, integrity-поля и NVMe-оптимизации. После размещения всех необходимых полей не остаётся места для inline data, project ID, FILE_ATTRIBUTE_*, Alternate Data Streams, reparse point data.

Рекомендуемый размер: **512 байт**. Это позволяет: inline data до ~108B (для файлов до 108 байт — нулевая фрагментация), inline ACL для простых случаев (24B), inline Owner/Group SID (по 16B), B+ tree root для extent map (96B inline), поля шифрования (algo + key_id + nonce + version = 40B),

Merkle root hash (32B), NVMe optimization fields, ADS count + ADS root.

Direct/indirect pointers категорически недопустимы. Только extent-based B+ дерево. Extent entry: 12-16 байт (logical_start 32-bit, phys_start 48-bit, length 16-bit, flags 16-bit). До 8 extent-ов inline в inode, далее — B+ дерево в отдельных блоках.

1.4 Журналирование

Предложен WAL (Write-Ahead Logging) — правильный базис. Но нужны уточнения. Рекомендуется **гибридный подход**: метаданные — всегда через WAL (журналируем все мутации), данные — CoW для файлов с флагом CoW + Ordered для остальных. Три режима журналирования: writeback (metadata only), ordered (default, data before metadata commit), journal_all (full ACID, для реестра и критичных файлов).

Размер журнала: автоматический = $\min(\text{volume_size} / 256, 4\text{GB})$, минимум 32MB для SSD/NVMe, рекомендовано 256MB-1GB. Структура: circular buffer с transaction descriptor + data blocks + commit record (CRC32C checksum). Replay: сканирование от head к tail, отбрасывание незавершённых транзакций, параллельная запись на NVMe.

1.5 Размер блока

4KB как единственный размер — **субоптимален**. Рекомендуется двухуровневая система: block_size=4KB (физический, атомарная запись, совместим с x86 page) + cluster_size (единица выделения, переменный: 4KB/16KB/64KB). На NVMe физический page = 16-32KB, erase block = 4-16MB, оптимальный write align = 64-128KB. 4KB random writes вызывают read-modify-write внутри SSD.

Решение: cluster_size = 16KB (default для NVMe), write combining для группировки 4KB writes, NVMe Write Zeroes для sparse extent-ов, TRIM/DISCARD на границах erase blocks.

1.6 Масштабируемость

Block groups **не масштабируются** до 100TB+. Том 100TB: 100,000 групп, 3.2GB только bitmap overhead, fsck = часы. С extent-based: Allocation B+ tree ($O(\log N)$ поиск), динамические inode-ы (не привязаны к группе), параллельный fsck по subtree-ам. Для миллиарда файлов: inode таблица как B+ дерево (не массив!), ACL deduplication (большинство ACL идентичны), SID-inode lookup через Bloom filter + hash table. Для multi-petabyte: 48-bit block pointers (до 4EB), 64-bit inode numbers.

1.7 NVMe-производительность

Per-CPU submission queues (2 на CPU: read/write separation), completion queues (1 на CPU, polling mode), queue depth 256-1024. IO_uring-подобный интерфейс: submission ring + completion ring, inode-based submission (нет path-lookup-open), registered buffers (zero-copy), linked operations. Zero-copy: O_DIRECT + registered buffers для DMA, mmap() с MAP_SHARED, sendfile()/splice() эквиваленты. Write combining buffer для группировки 4KB в 16KB aligned writes. Целевой WAF: sequential 1.1-1.3, random 1.5-2.5.

1.8 HDD-производительность

Предложенный дизайн **не упоминает HDD вообще** — это ошибка. Внешние HDD распространены для backup/archive. Решения: rotational awareness flag в superblock (auto-detect через NVMe IDENTIFY vs SCSI INQUIRY), seek-optimized placement (cylinder groups), aggressive readahead (1MB+), larger cluster_size (64KB), directory B+ tree ($O(\log N)$ seeks), journal на начале диска.

1.9 Большие файлы

Extent-based подход идеально подходит для файлов 100GB+. Contiguous file = 1 extent (16B inline), moderately fragmented (50 extents) = 800B inline, heavily fragmented (10K extents) = B+ tree depth=1 (~30 leaf nodes). Нужны: preallocation API (PREALLOC_ZEROED, PREALLOC_UNINIT для SetFileValidData compat), native sparse file support через extent map holes.

1.10 Мелкие файлы

Inline data обязателен. Без inline data: 10M файлов по 1KB = 45GB (67% waste). С inline data: 5GB (9x меньше, 2x меньше IOPS). Directory B+ tree index: SipHash-2-4 (DoS-resistant), ~170 entries per 4KB leaf, depth=1 handles ~29K файлов, depth=2 ~5M файлов. NT compat: NtQueryDirectoryFile, FILE_DIRECTORY_INFORMATION, 8.3 short names.

1.11 Снапшоты и Copy-on-Write

Per-filesystem CoW — **ошибочная архитектура** (проблема ZFS/Btrfs). POLER-FS должен использовать **per-file CoW** с гибким переключением. CoW OFF (default): in-place write + journal protection — для БД, логов, временных файлов. CoW ON: автоматически для VM disk images, snapshotted directories, registry hives. Extent-level CoW (рекомендуемая гранулярность), refcount tree (только shared blocks, refcount>1), интеграция с VSS API Windows.

1.13 Дедупликация

Offline dedup как primary, optional inline для VM workloads. Inline dedup добавляет неприемлемую latency (~50us на NVMe budget ~100us). Offline: Phase 1 — сканирование с BLAKE3 hash + Cuckoo filter (~3GB на 1TB), Phase 2 — batched dedup с journal protection. Per-directory opt-in для inline dedup.

1.14 Шифрование

Два уровня: (1) Per-volume (BitLocker equivalent) — AES-256-XTS, transparent для ФС, HW acceleration AES-NI. (2) Per-inode (EFS equivalent) — AES-256-XTS для disk, AES-256-GCM для новых файлов, ChaCha20-Poly1305 как fallback (нет HW AES). **Критическая ошибка:** ChaCha20 для disk encryption — ПЛОХО (stream cipher: random access требует recompute; нет wide-block). Filename encryption: AES-256-CTS — хороший выбор. Key hierarchy: Master Key (TPM 2.0 sealed) → Volume Key → File Key (HKDF per-inode).

1.15 Контроль целостности

Per-inode SHA-256 integrity — **недостаточен**. Нужен: per-block checksum (BLAKE3-128 truncated, 16B в block trailer) + Merkle tree (BLAKE3-256, root hash в inode). BLAKE3 в 3-5x быстрее SHA-256, имеет native Merkle tree support, поддерживает keyed mode (MAC). SHA-256 как fallback для FIPS compliance. Block layout: Data (4072B) + Checksum (24B) = 4096B. Scrub: фоновый traversal B+ tree с BLAKE3 верификацией.

1.16 NT-совместимые ACL

NT ACL — правильный выбор, но реализация требует: **ACL deduplication** (большинство файлов имеют идентичный наследуемый ACL, храним один раз → ссылка по ID), **SACL support** (audit ACL — критически важен для Active Directory, compliance, anti-cheat), **ACE inheritance** (4 флага: OI, CI, IO, NP), **ACL cache** (hash table: acl_id → parsed ACL, LRU eviction, per-CPU RCU-protected lookup), **VFS translation layer** (NT ACL для POLER-FS, POSIX ACL для ext2, None для FAT32).

1.17 Совместимость с Windows API

Самый критичный аспект POLER-FS. Предложенный дизайн практически не рассматривает: Alternate Data Streams (ADS) — множественные \$DATA атрибуты с именами, каждый ADS — полный «файл» с ACL, size, extent map; Reparse Points — символические ссылки, junction points, mount points, UWP APPEXECLINK; Mandatory file locking (не advisory как POSIX!); Oplocks (Opportunistic Locks) — критичны для SMB/local anti-cheat мониторинга. Без этих механизмов POLER-OS не сможет обеспечить NT совместимость.

1.18 Восстановление после сбоя

Целевое время: NVMe <200ms, HDD <5s. Оптимизации: checkpointing (не replay весь журнал, только после последнего checkpoint, интервал 5-30 сек), parallel replay (multi-queue NVMe), lazy orphan cleanup (mark + background free), soft recovery mode (mount RO сразу, replay в фоне). Orphan inode list: при создании файла inode добавляется в orphan list, при создании dentry — удаляется. Crash после allocation — orphan cleanup при mount.

1.19 Стратегия кэширования

Multi-level: L1 CPU caches (inode hot fields, dentry cache, ACL parsed), L2 Page Cache (inode cache 2% RAM, dentry cache 1%, page cache 40-60%), L3 NVMe. Write policy: file data — write-back + ordered flush, metadata — write-through via journal, encryption keys — write-through + zeroize on evict. Dirty page flush: 10% RAM → background flush, 20% → aggressive, 40% → all blocked, dirty age >30s → flush. NT file flags: WRITE_THROUGH, NO_BUFFERING, SEQUENTIAL_SCAN, RANDOM_ACCESS.

1.20 Организация VFS

VFS abstraction layer: NT API → I/O Manager (IRP routing, Fast I/O path, async I/O) → VFS → filesystem drivers. VFS operations: mount/umount, lookup, create/unlink, read/write, readdir, ioctl, mmap, truncate, lock. Mount points, union mounts. VFS ACL translation: POLER-FS (NT ACL) ↔ VFS ACL ↔ ext2 (POSIX ACL) / FAT32 (None→Everyone:Full).

2. Сравнение POLER-FS с 12 файловыми системами

Ниже приведён сравнительный анализ POLER-FS с 12 существующими файловыми системами. Для каждой указаны сильные и слабые стороны, что стоит перенять и что категорически не стоит повторять.

FAT32

Сильные стороны: Универсальная совместимость (каждая платформа); минимальная реализационная сложность (~2000 строк); предсказуемая латентность; нулевой оверхед метаданных; UEFI ESP поддержка.

Слабые стороны: Жёсткий предел 4 ГБ на файл; катастрофическая фрагментация; нет целостности; нет безопасности; неэффективное использование пространства.

Что перенять: Самодостаточный superblocK (BPB содержит всё для монтирования); emergency recovery mode без зависимости от B+ tree.

Не повторять: Отсутствие журналирования; линейный поиск в каталогах; цепочечная аллокация; фиксированные размеры полей.

exFAT

Сильные стороны: 64-bit DataLength; NoFatChain оптимизация; Up-case table для case-insensitive; TexFAT (опциональный).

Слабые стороны: Нет журналирования по умолчанию; нет integrity; фрагментация без инструментов; ограниченные таймстампы.

Что перенять: NoFatChain концепт (optimized path для contiguous allocation); Up-case table для case-insensitive NT compat.

Не повторять: Опциональное журналирование (= никакого в реальности); отсутствие checksumming данных.

NTFS

Сильные стороны: MFT атрибутная модель (всё есть атрибут); ADS; Reparse Points; NT ACL; B-tree каталоги; USN Journal; LFS двухфазный commit; sparse files.

Слабые стороны: Фрагментация MFT; устаревший I/O path (single queue); нет CoW; нет checksumming данных; секретность реализации; проблемы с большими каталогами.

Что перенять: Атрибутную модель MFT; Reparse Points; полную модель NT ACL (DACL/SACL/ACE/inheritance/integrity levels); USN Journal (change journal).

Не повторять: In-place overwrite без CoW; отсутствие integrity данных; single-queue I/O; зависимость от дефрагментации.

ext2

Сильные стороны: Простота (~8000 строк); быстрый fsck (параллелизация по BG); block group locality; stable on-disk format с 1993; sparse superblocK; online resize.

Слабые стороны: Нет журналирования; косвенные блоки; фиксированный inode (128B); нет extents; нет CoW/снапшотов; нет шифрования.

Что перенять: Пространственную локальность (allocation domains); sparse metadata redundancy; простоту fsck.

Не повторять: Косвенные блоки; отсутствие журналирования; фиксированный inode размер.

ext3

Сильные стороны: Обратная совместимость с ext2; три режима журналирования; HTree-индексация каталогов; быстрое recovery; external journal.

Слабые стороны: Архитектура ext2 + журнал; serial journal (bottleneck); нет extent allocation; нет delayed allocation; 16TB предел.

Что перенять: Три режима журналирования; external journal на отдельном устройстве.

Не повторять: Serial WAL при параллельных AG; журнал поверх устаревшей архитектуры; поблочную аллокацию без delayed allocation.

ext4

Сильные стороны: Extent-based allocation; delayed allocation (delalloc); mballoc; journal checksumming; bigalloc; inline data (<60B); fast commit; project quotas.

Слабые стороны: Extent tree не балансируется; нет CoW; нет integrity данных; ограниченная inline data; 16TB предел файла; нет встроенного шифрования.

Что перенять: Delayed allocation + mballoc; journal checksumming; inline data; fast commit; bigalloc-style clustering.

Не повторять: Несбалансированное extent tree; отсутствие end-to-end integrity; in-place overwrite.

XFS

Сильные стороны: Allocation Groups (параллелизм без блокировок); B+ дерево везде; delayed allocation; space preallocation; Direct I/O optimised; scalability до 500TB+; reverse mapping (rmapbt); online fsck; reflink (block-level CoW).

Слабые стороны: Невозможно shrink; медленный metadata-heavy workload; serial WAL; нет integrity данных; сложность формата.

Что перенять: Allocation Groups/domains (привязанные к NVMe queue); B+ tree для всего; reverse mapping; CoW extent size hints; preallocation API; online fsck через rmap.

Не повторять: Serial WAL при параллельных AG; невозможность shrink; aggressive delayed allocation без гарантии fsync.

Btrfs

Сильные стороны: CoW-дерево (O(1) snapshot); per-file CoW; checksumming данных и метаданных (CRC32c); прозрачное сжатие; дедупликация; подтома; send/receive; RAID integration; online defrag.

Слабые стороны: CoW = фрагментация (catastrophic для БД); RAID5/6 ненадёжны; ENOSPC при 30% свободного; медленный fsck; global transaction lock; нестабильность (до ~5.10 kernel).

Что перенять: CoW для данных + WAL для метаданных; per-extent checksumming; subvolumes с Space Sharing; send/receive; прозрачное сжатие (zstd).

Не повторять: CoW-индуцированную фрагментацию; global transaction lock; ENOSPC при свободном месте; RAID на уровне ФС.

ZFS

Сильные стороны: End-to-end integrity (Merkle tree); CoW transaction model; RAID-Z; Storage Pools; O(1) snapshots+clones; send/receive; ARC+L2ARC; ZIL (SLOG); native encryption; scrub; dataset hierarchy.

Слабые стороны: Memory hunger (50% RAM); write amplification (3-5x); CDDL лицензия; не shrinkable; dedup опасна; медленный pool import; фрагментация при >80%.

Что перенять: Merkle tree integrity; dataset/subvolume с наследованием; ZIL-подобный SLOG; scrub; native encryption per-dataset; separate intent log device.

Не повторять: Memory hunger; dedup по умолчанию; CDDL лицензирование; невозможность shrink; неконтролируемую CoW write amplification.

APFS

Сильные стороны: CoW оптимизированная для SSD; Space Sharing; Clone files (zero-cost); per-file encryption (AES-256-XTS); snapshots; nanosecond timestamps; inline data (~8KB).

Слабые стороны: Закрытая реализация; нет integrity данных; нет RAID; фрагментация CoW; медленный directory enumeration; нет dedup.

Что перенять: Per-file encryption с key hierarchy (volume key → file keys); Space Sharing / Container model; clone files API; band allocation (уменьшение CoW фрагментации); inline data до 8KB.

Не повторять: Отсутствие checksumming данных; закрытость; отсутствие redundancy.

F2FS

Сильные стороны: Спроектирована для NAND flash; multi-head logging (6 типов); adaptive logging (LFS/SSR); inline data (~3.4KB); ATGC; pin file sections для DB.

Слабые стороны: LFS = фрагментация; GC overhead при >80%; нет CoW; нет integrity; нет ACL модели; checkpoint overhead; нет snapshot.

Что перенять: Multi-head logging/separation по temperature; NAND-aware alignment; adaptive mode switching; pin file API; extent cache in-memory.

Не повторять: LFS-подход (GC overhead); отсутствие integrity; checkpoint-based consistency; segment-based GC.

ReFS

Сильные стороны: Block Cloning (zero-cost); Integrity Streams (checksumming + auto-repair); Storage Spaces integration; mirroring с auto-correct; Sparse VDL.

Слабые стороны: Только Windows; нет шифрования; нет dedup (удалена); деградация производительности; нет compression; не bootable; устаревание.

Что перенять: Integrity Streams (per-file checksumming + auto-repair при redundancy); Block cloning API; auto-heal при mirrored storage; Sparse VDL.

Не повторять: Проприетарность; отсутствие шифрования; зависимость от VSS для snapshots; удаление фич после релиза.

2.13 Сводная таблица сравнения

ФС	Аллокация	Журнал	CoW	Шифр.	Integ.	ACL	Снапш.
FAT32	Цепочка	Нет	Нет	Нет	Нет	Нет	Нет
exFAT	Цепочка+	Нет	Нет	Нет	Нет	Нет	Нет
NTFS	Экстенты	WAL	Нет	EFS	Нет	NT	VSS
ext2	Косвенные	Нет	Нет	Нет	Нет	POSIX	Нет
ext3	Косвенные	WAL	Нет	Нет	Нет	POSIX	Нет
ext4	Экстенты	WAL	Нет	fsencrypt	Нет	POSIX	Нет
XFS	B+tree	WAL	Reflink	Нет	Нет	POSIX	Reflink
Btrfs	CoW tree	CoW	Да	Нет	CRC32c	POSIX	O(1)

ФС	Аллокация	Журнал	CoW	Шифр.	Integ.	ACL	Снапш.
ZFS	CoW record	CoW+ZIL	Да	AES-GCM	Merkle	NFSv4	O(1)
APFS	CoW tree	CoW	Да	AES-XTS	Нет	POSIX	O(1)
F2FS	LFS segment	Checkpoint	Нет	Нет	Нет	POSIX	Нет
ReFS	B+tree	CoW	Нет	Нет	Опц.	NT	VSS
POLER-FS	B+tree ext	WAL	Per-file	AES-XTS	BLAKE3	NT	CoW

3. Юридический анализ проекта

3.1 Совместимость API — законна ли реализация Windows API?

Да, законна. Ключевые прецеденты: **Oracle v. Google (2021)** — Верховный суд США постановил, что API интерфейсы являются «методом операции» (method of operation), и их копирование для обеспечения совместимости constitutes fair use. Это прямой прецедент для POLER-OS. **Lotus v. Borland (1996)** — суд постановил, что интерфейсные элементы (имена, сигнатуры, меню) не охраноспособны авторским правом. **Sony v. Connectix (2000)** — эмуляция коммерческого продукта законна, даже если требует reverse engineering. **Директива ЕС 2009/24/EC, Статья 6** — прямо разрешает декомпиляцию и reverse engineering для обеспечения совместимости, если информация не была ранее доступна иными средствами.

Реализация Windows NT API от третьего лица полностью подпадает под установленные юридические прецеденты. API интерфейсы (имена функций, сигнатуры, структуры параметров) не защищены авторским правом, когда их использование трансформативно — то есть создаёт новую платформу, а не копирует существующую.

3.2 Clean-room разработка

Clean-room design — юридически признанный метод, при котором команда спецификации (изучает поведение оригинала и пишет спецификацию) отделена от команды реализации (пишет код исключительно по спецификации, не имея доступа к оригинальному коду). В случае одного разработчика рекомендуется **модифицированная чистая комната**: документирование каждого решения через black-box анализ (вход-выход), фиксация sources of information для каждого API, запрет на использование leak-источников, использование только публичной документации (MSDN, Windows SDK headers, ReactOS wiki).

3.3 Реализация поведения ОС

Реализация **поведения** ОС (например, возврат NtQuerySystemInformation) законна, если это не копирование кода. Ключевое различие: поведение (функция возвращает X при входе Y) не охраноспособно; реализация (конкретный код, вычисляющий X из Y) охраноспособна. POLER-OS реализует поведение через собственный код — это законно.

3.4 Обратная совместимость

Обратная совместимость сама по себе законна. Однако **патенты на NTFS и ReFS** — реальный риск. Решение: POLER-FS использует собственную файловую систему, а не реализует NTFS. Это полностью исключает патентные риски, связанные с NTFS/ReFS. Формат POLER-FS разработан с нуля и не нарушает патенты Microsoft на файловые системы.

3.5 Риски нарушения интеллектуальной собственности

Тип риска	Уровень	Описание	Митигация
Copyright (код)	Низкий	Копирование исходного кода Windows	Clean-room: собственный код с нуля
Патенты (ФС)	Средний	Патенты Microsoft на NTFS/ReFS	POLER-FS вместо NTFS; свой формат
Патенты (API)	Низкий	Патенты на конкретные алгоритмы	Альтернативные реализации поведения
Товарные знаки	Низкий	Использование "Windows"	Не называть POLER-OS "Windows"
Коммерческая тайна	Низкий	Использование leak-источников	Только black-box анализ

3.6 Интерфейс vs Реализация

Критическое юридическое различие. Интерфейс (имена функций, сигнатуры, форматы структур, протоколы) — не защищён авторским правом (Oracle v. Google). Реализация (конкретный код, алгоритмы, данные) — защищена. POLER-OS реализует интерфейсы Windows API (законно) через собственный код (законно), не копируя реализацию Microsoft. Это аналогично тому, как ReactOS, Wine и CrossOver работают легально уже десятилетиями.

4. Архитектура совместимости с Windows

4.1 PE Loader

PE (Portable Executable) Loader — компонент, загружающий .exe файлы. Должен: парсить DOS header → PE header → Optional Header → Section Table; загружать секции в память с правильными адресами и правами; обрабатывать imports (IAT resolution); инициализировать TLS; передавать управление в Entry Point. Anti-cheat проверяет: совпадение загруженного образа с файлом на диске (хеш секций), целостность IAT, корректность PEB->ImageBaseAddress. Сложность: 8/10, ~10-15 недель.

4.2 DLL Loader

DLL Search Order: Known DLLs (предзагруженные из System32), AppPath, SxS manifests (side-by-side assembly versioning), API Set mapping (api-ms-win-*.dll → реальная DLL). Каждый DLL имеет собственный PE header, imports, TLS. Сложность: 9/10, ~15-20 недель. Anti-cheat проверяет: загрузку только из ожидаемых путей, целостность Known DLLs.

4.3 Object Manager

NT Object namespace — иерархия объектов: Device, BaseNamedObjects, Registry, KernelObjects. Каждый объект имеет тип, security descriptor, reference count. Symbolic links (C: → DeviceHarddiskVolume1). Anti-cheat использует NtOpenDirectoryObject/NtQueryDirectoryObject для перечисления объектов. Сложность: 7/10, ~8-12 недель.

4.4 Registry

Полное дерево HKLM_SYSTEM... не нужно. Античиты читают: HKLM_SYSTEM_CurrentControlSet_Services (драйверы), HKLM_SOFTWARE_Microsoft_Windows_NT_CurrentVersion (версия ОС), HKLM_SYSTEM_CurrentControlSet_Control_Session Manager (переменные среды), HKLM_SOFTWARE_Microsoft_Cryptography (машино-GUID). Нужен: in-memory registry + VFS-backed hive files, NtOpenKey/NtQueryValueKey/NtEnumerateKey. Сложность: 6/10, ~6-8 недель.

4.5 Security Reference Monitor

Token модель: Access Token (SID пользователя, группы, привилегии, integrity level), impersonation, SeDebugPrivilege, SeLoadDriverPrivilege. Privilege check: SeAccessCheck(). Integrity levels: Untrusted → Low → Medium → High → System. Anti-cheat проверяет привилегии текущего процесса и integrity level. Сложность: 7/10, ~8-10 недель.

4.6 ALPC

Advanced Local Procedure Call — основной IPC механизм Windows. Используется csrss.exe, сервисами, COM. Anti-cheat общается с сервисом через ALPC. Нужен: NtConnectPort, NtAlpcSendWaitReceivePort, ALPC port object в Object Manager. Сложность: 7/10, ~10-16 недель. LPC — legacy, не обязателен (заменён ALPC).

4.7 Scheduler и Memory Manager

Scheduler: Thread priorities (0-31, real-time 16-31), priority boost для I/O, affinity mask, quantum (variable/fixed), IRQL implications (DISPATCH_LEVEL = scheduler lock). Anti-cheat не проверяет scheduler напрямую, но замечает аномалии (неправильные приоритеты). Сложность: 8/10,

~12-16 недель.

Memory Manager: VAD (Virtual Address Descriptor) tree, section objects (shared memory), page fault handling, copy-on-write pages, address space layout. NtAllocateVirtualMemory, NtMapViewOfSection, NtProtectVirtualMemory. Anti-cheat сканирует VAD на предмет injected DLL. Сложность: 9/10, ~16-20 недель.

4.8 Драйверная модель

Невозможно загрузить .sys драйверы. Windows kernel drivers скомпилированы для ntoskrnl.exe и вызывают его internal functions. POLER-OS не может предоставить бинарно-совместимый ntoskrnl. Решение: собственный драйверный формат (POLER Driver Interface), драйверы пишутся под POLER-OS API, shim-слой для популярных IOCTL. Это исключает загрузку kernel-mode античитов.

4.9 Системные вызовы, User mode, Kernel mode

Системные вызовы: Windows syscall numbers меняются между версиями. Стратегия: ntdll.dll shim перехватывает все syscall и маршрутизирует в POLER-OS native syscall table. Приложение вызывает Win32 API → ntdll.dll shim → POLER-OS kernel.

User mode: ntdll.dll (shim, ~2000+ функций), csrss.exe (подсистема Win32), kernel32.dll, user32.dll, gdi32.dll. Каждый процесс имеет PEB (Process Environment Block) и TEB (Thread Environment Block).

Kernel mode: Структуры EPROCESS, ETHREAD должны быть реализованы с правильными offset-ами для NtQuerySystemInformation. Остальные kernel structures могут отличаться.

5. Античит-совместимость: технический анализ

5.1 Принцип: корректная реализация, не обход

POLER-OS не обманывает античиты — она предоставляет им то, что они ожидают найти: полностью реализованную Windows-совместимую среду, написанную с нуля. Разница между «хаком» и «реализацией»: хак — `if(anti_cheat_check) return TRUE` (обман); реализация — полноценный `NtQuerySystemInformation()`, который возвращает правильные данные, потому что система реально работает как Windows.

5.2 Что решается корректной реализацией NT API

Проверка античита	Что проверяет	Реализация POLER-OS	Статус
PEB/TEB integrity	Структуры процесса/потока	Полная реализация PEB/TEB	Решаемо
NtQuerySystemInfo	Список процессов, модулей	Native syscall через POLER	Решаемо
EPROCESS/ETHREAD	Kernel структуры	Полная реализация структур	Решаемо
Object Manager	Device, BaseNamedObjects	Полный NT Object namespace	Решаемо
Registry queries	Версия ОС, драйверы, GUID	Registry с правильными ключами	Решаемо
IAT integrity	Import Address Table	Корректный PE loader	Решаемо
Heap integrity	Консистентность кучи	NT heap allocator	Решаемо
File system checks	Пути, ACL, file attributes	POLER-FS с NT ACL	Решаемо
User-mode debugging	Отладочные API	Correct Dbg* API behaviour	Решаемо

5.3 Что НЕ решается корректной реализацией

Проблема	Причина	Тип ограничения
Kernel-mode .sys драйверы	Скомпилированы для ntoskrnl.exe	Техническое
WHQL подпись	Требуется Microsoft certification	Политика
VBS/HVCI	Требуется Hyper-V гипервизор	Техническое
TPM attestation	Microsoft PKI, не наш сертификат	Политика
ELAM (Early Launch AM)	Требуется Microsoft-signed драйвер	Политика
Secure Boot chain	Microsoft UEFI CA	Политика

5.4 Совместимость по конкретным античитам

Античит	Тип	Шанс совместимости	Причина
VAC (Valve)	User-mode	Высокий	Только user-mode проверки
EQU8	Преим. user-mode	Средний	User-mode + опциональный kernel
EAC (EasyAntiCheat)	Kernel driver	Низкий	Требуется загрузка .sys драйвера
BattlEye	Kernel driver	Низкий	Агрессивные kernel проверки
Vanguard (Riot)	Boot-start driver + HVCI	Очень низкий	Требуется HVCI + TPM

5.5 Реальный путь к kernel-mode античитам

Единственный реальный путь к поддержке kernel-mode античитов (EAC, BattlEye, Vanguard) — **сотрудничество с их разработчиками**. Это означает: портирование античита на POLER-OS kernel API (полный релиз SDK), предоставление собственного Anti-Cheat SDK для разработчиков игр, сертификационную программу для POLER-OS-совместимых античитов. Это стандартный путь: Sony и Nintendo требуют от разработчиков античитов портирование

под их платформы. POLER-OS должна действовать аналогично.

Без сотрудничества с разработчиками kernel-mode античитов — эти античиты НЕ будут работать на POLER-OS. Это вопрос политики, не техники.

6. Roadmap развития проекта

Полная дорожная карта от текущего состояния до релиза 1.0, с оценками для одного разработчика.

Фаза	Описание	Зав.	Приор.	Слож.	Трудоём.	Риски
1. Минимальное ядро	Boot, GDT, IDT, paging	-	P0	6	8-12 нед.	Аппаратные баги QEMU
2. Управление памятью	PMM, VMM, heap, slab	1	P0	7	10-14 нед.	Фрагментация, leaks
3. SMP	APIC, IPI, TLB shutdown	1,2	P0	9	12-16 нед.	Data corruption, deadlocks
4. Планировщик	Потоки, процессы, приоритеты	2,3	P0	7	8-12 нед.	Priority inversion
5. Синхронизация	Spinlocks, mutexes, RCU	3	P0	6	6-8 нед.	Deadlocks, starvation
6. Файловые системы	VFS, ext2 (r/w)	2,4,5	P1	7	8-10 нед.	Data corruption
7. Драйверная модель	PCI, USB, AHCI	2,3,5	P1	8	10-14 нед.	Железо-специфичные баги
8. Сеть	TCP/IP, sockets	4,5,6	P1	8	12-16 нед.	Стек-уязвимости
9. Графика	Framebuffer, compositor	2,4,6	P2	7	8-10 нед.	GPU-specific
10. Vulkan GPU	ICD driver	7,9	P1	10	30-40+ нед.	CRITICAL: спецификация сложна
11. User space	ELF loader, libc, shell	4,5,6	P1	7	10-14 нед.	libc completeness
12. VFS complete	POLER-FS basic	6	P1	8	12-16 нед.	Формат стабильность
13. Win32 subsystem	PE/DLL loader, basic Win32	2,11	P1	9	20-30 нед.	CRITICAL: огромный API
14. NT API	ntdll, ObjMgr, Registry, SRM	3,11	P1	10	30-40+ нед.	CRITICAL: ~2000 функций
15. App compat	Первые Windows-приложения	4,11	P2	8	12-20 нед.	Edge cases бесконечны
16. Dev tools	Compiler, debugger, build	11	P2	6	10-14 нед.	Инструмент chain
17. Testing	CI/CD, fuzzing, regression	6+	P2	7	8-12 нед.	Coverage
18. Release stages	Alpha → Beta → 1.0	All	P0	-	-	Polish, documentation

Итого без упрощений: ~252 недели (4.8 года для одного разработчика). С предлагаемыми упрощениями (Mesa3D fork вместо своего Vulkan, shim вместо kernel-native Win32, ext4+encryption вместо POLER-FS с нуля, lwIP вместо своего TCP/IP, musl port вместо своего libc) — экономия ~108 недель (2 года), итог: ~3-4 года до работающей beta.

7. Критический аудит проекта

7.1 Слабые места

#	Слабое место	Влияние	Серьёзность
1	TLB shutdown отложен	SMP data corruption гарантирован	CRITICAL
2	MAX_TASKS=8	Неприемлемо для реальной нагрузки	CRITICAL
3	Нет AHCI/NVMe драйвера	OS работает только в QEMU	CRITICAL
4	Shim vs kernel-native не решено	Архитектурное противоречие	HIGH
5	Immutable kernel + anti-cheat	Несовместимые цели	HIGH
6	Собственная криптография без peer review	Небезопасно	HIGH
7	Один разработчик	Single point of failure	MEDIUM
8	Нет автоматического тестирования	Регрессии не детектируются	MEDIUM

7.2 Возможные архитектурные ошибки

1. Попытка быть «полноценным Windows» вместо «достаточным Windows». ReactOS потратила 20+ лет, пытаясь клонировать Windows полностью, и всё ещё не готова. POLER-OS должна сфокусироваться на конкретных сценариях (игры, офисные приложения) и реализовать только необходимый минимум API для них.

2. Kernel-native Win32 вместо userspace shim. Реализация Win32 в ядре (как в Windows) создаёт огромную поверхность атаки и требует в 3-5 раз больше работы. Userspace shim (как в Wine) — проще, безопаснее, быстрее в разработке. Компромисс: shim для большинства функций, kernel-native только для критичных path (memory management, process creation).

3. Собственная криптография. PND v8 и POLER-CTR без peer review — insecure. Следует использовать стандартные алгоритмы: AES-256-GCM (FIPS-approved), ChaCha20-Poly1305 (IETF standard), с библиотеками, прошедшими аудит.

7.3 Альтернативные решения

Текущий план	Альтернатива	Преимущество	Экономия
Свой Vulkan driver	Mesa3D fork	Battle-tested code, community	~20 нед.
Kernel-native Win32	Userspace shim (Wine-like)	Проще, безопаснее, быстрее	~30 нед.
POLER-FS с нуля	ext4 + encryption layer	Стабильный формат	~10 нед.
Свой TCP/IP стек	lwIP port	Минимальный, проверенный	~10 нед.
Свой libc	musl port	Стандартный, полный	~6 нед.
PND + POLER-CTR	AES-GCM + ChaCha20	Peer-reviewed стандарты	~4 нед.

7.4 Масштаб проекта

Для сравнения: Linux kernel — ~30М строк кода, 1700+ разработчиков; ReactOS — ~12М строк, 20+ лет, всё ещё alpha; FreeBSD — ~14М строк, сотни разработчиков. POLER-OS для минимальной работоспособности (запуск notepad.exe) — ~500K-1М строк кода. Для игровой совместимости (запуск Гэнсин) — ~3-5М строк. Это реалистично для 3-5 разработчиков за 4-6 лет, или 1 разработчика за 8-12 лет.

7.5 Реалистичные vs Многолетние компоненты

Компонент	Реалистичность	Время	Условие
Ядро + SMP + память	Реалистично	1-2 года	Один разработчик
VFS + ext2	Реалистично	0.5-1 год	
PE/DLL Loader	Реалистично	0.5-1 год	
Basic Win32 (window, input)	Реалистично	1-2 года	
Vulkan GPU driver	Многолетний	3-5 лет	Без Mesa3D fork
Полный NT API (~2000 функц.)	Многолетний	5-8 лет	
Anti-cheat совместимость	Требует кооперации	Зависит	Сотрудничество с EAC/BE
Собственный POLER-FS	Многолетний	2-3 года	Полная реализация

7.6 Критические упрощения без потери архитектурной идеи

1. Mesa3D fork вместо своего Vulkan. Vulkan specification — 1800+ страниц. Написание conformant driver с нуля — 3-5 лет. Fork Mesa3D (open source, MIT-licensed) — 6-12 месяцев. Архитектурная идея (Vulkan-first GPU) сохраняется полностью.

2. Userspace shim вместо kernel-native Win32. Wine/Proton доказали, что userspace shim работает для тысяч приложений. POLER-OS может использовать аналогичный подход, но с собственным ядром вместо Linux. Архитектурная идея (Windows API через clean-room) сохраняется.

3. ext4 + encryption layer вместо POLER-FS с нуля. ext4 — battle-tested на миллионах серверов. Добавление encryption layer (fscrypt) + NT ACL translation в VFS — 3-6 месяцев вместо 2-3 лет. POLER-FS может быть добавлена позже, когда ядро стабильно.

Первая цель: запустить notepad.exe. Не Vulkan, не Гэнсин, не античиты. Самое простое Windows-приложение. Если notepad.exe работает — архитектура верна. Если нет — нужно пересмотреть подход.

С упрощениями — 3-4 года до работающей beta. Без упрощений — проект не доживёт. Архитектурная целостность сохраняется при любом пути: Vulkan-first, clean-room NT API, собственное ядро.

8. Заключение

Проведённый технический аудит подтверждает, что архитектурная концепция POLER-OS жизнеспособна, но требует значительных корректировок в реализации. Основные выводы:

Файловая система POLER-FS требует перехода от block groups к extent-based архитектуре с B+ деревом; увеличения inode до 512 байт; добавления inline data, ADS, reparse points, oplocks; реализации per-file CoW вместо per-filesystem; использования AES-256-XTS (не ChaCha20) для дискового шифрования; BLAKE3 для integrity вместо SHA-256.

Юридическая сторона проекта законна: реализация Windows API через clean-room разработку защищена прецедентами Oracle v. Google, Lotus v. Borland, и Директивой ЕС 2009/24/EC. Главный патентный риск (NTFS) полностью устранён использованием POLER-FS.

Совместимость с Windows реализуема для user-mode приложений через PE/DLL loader + ntddl.dll shim + NT API implementation. Kernel-mode драйверы (включая kernel-античиты) не могут быть загружены — это фундаментальное техническое ограничение, единственное решение — сотрудничество с разработчиками античитов.

Античит-совместимость для user-mode античитов (VAC, EQU8) достижима через корректную реализацию NT API. Для kernel-mode античитов (EAC, BattlEye, Vanguard) необходимо портирование под POLER-OS API — это вопрос политики и бизнес-партнёрств, а не техники.

Реалистичный timeline: 3-4 года до работающей beta с предлагаемыми упрощениями (Mesa3D fork, userspace shim, ext4+encryption, lwIP, musl). Без упрощений — проект не доживёт из-за масштаба. Первая цель: запустить notepad.exe.